



# **DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

Discover. Learn. Empower.

## **Experiment - 7**

**Student Name:** Jayesh Saini

**UID:** 23BCS11619

**Branch:** BE-CSE

**Section/Group:** KRG\_2A

**Semester:** 6<sup>th</sup>

**Date of Performance:** 24/3/26

**Subject Name:** System Design

**Subject Code:** 23CSH-314

### **Aim:**

To design and analyze a scalable ride-sharing application that enables users to book rides and drivers to accept and fulfill requests in real time, ensuring high availability, low latency, and reliability at large scale.

### **Objectives:**

1. To understand system design principles for large-scale ride-sharing systems.
2. To design APIs for ride booking, driver matching, and payment services.
3. To identify functional and non-functional requirements.
4. To apply distributed system concepts such as CAP theorem.
5. To design a high-availability and low-latency system architecture.

### **Procedure-**

1. Identify functional requirements of a ride-sharing application.
2. Define non-functional requirements such as scalability, latency, and availability.
3. Analyze CAP theorem trade-offs for real-time ride matching.
4. Identify core entities such as Rider, Driver, Ride, and Payment.
5. Design the system architecture using Draw.io.
6. Validate the design for real-time ride booking and tracking scenarios.

### **Functional Requirements –**

#### **User Management**

1. User should be able to register.
2. User should be able to login/logout.

3. User should be able to update profile information.

#### Ride Booking

1. User should be able to enter pickup and drop location.
2. System should provide fare estimation.
3. User should be able to request a ride.
4. User should be able to choose ride type (car, bike, etc.).

#### Driver Management

1. Driver should be able to register and login.
2. Driver should be able to accept/reject ride requests.
3. Driver should be able to update availability status.

#### Ride Management

1. System should match riders with nearby drivers.
2. System should provide real-time ride tracking.
3. System should maintain ride history.
4. System should allow cancellation of rides.

#### Payment & Rating

1. System should support online/offline payments.
2. System should generate invoices after ride completion.
3. User should be able to rate drivers.
4. Driver should be able to rate users.

### **Non-functional Requirements-**

#### Scalability

1. Target Users: 1.5 Million users and drivers
2. Concurrent ride requests: High during peak hours
3. System should scale horizontally

#### Availability (CAP Theorem)

1. System should prefer High Availability and Partition Tolerance.
2. Follow Eventual Consistency for ride status updates.

#### Latency

1. Ride matching latency: < 2 seconds
2. Location update latency: near real-time

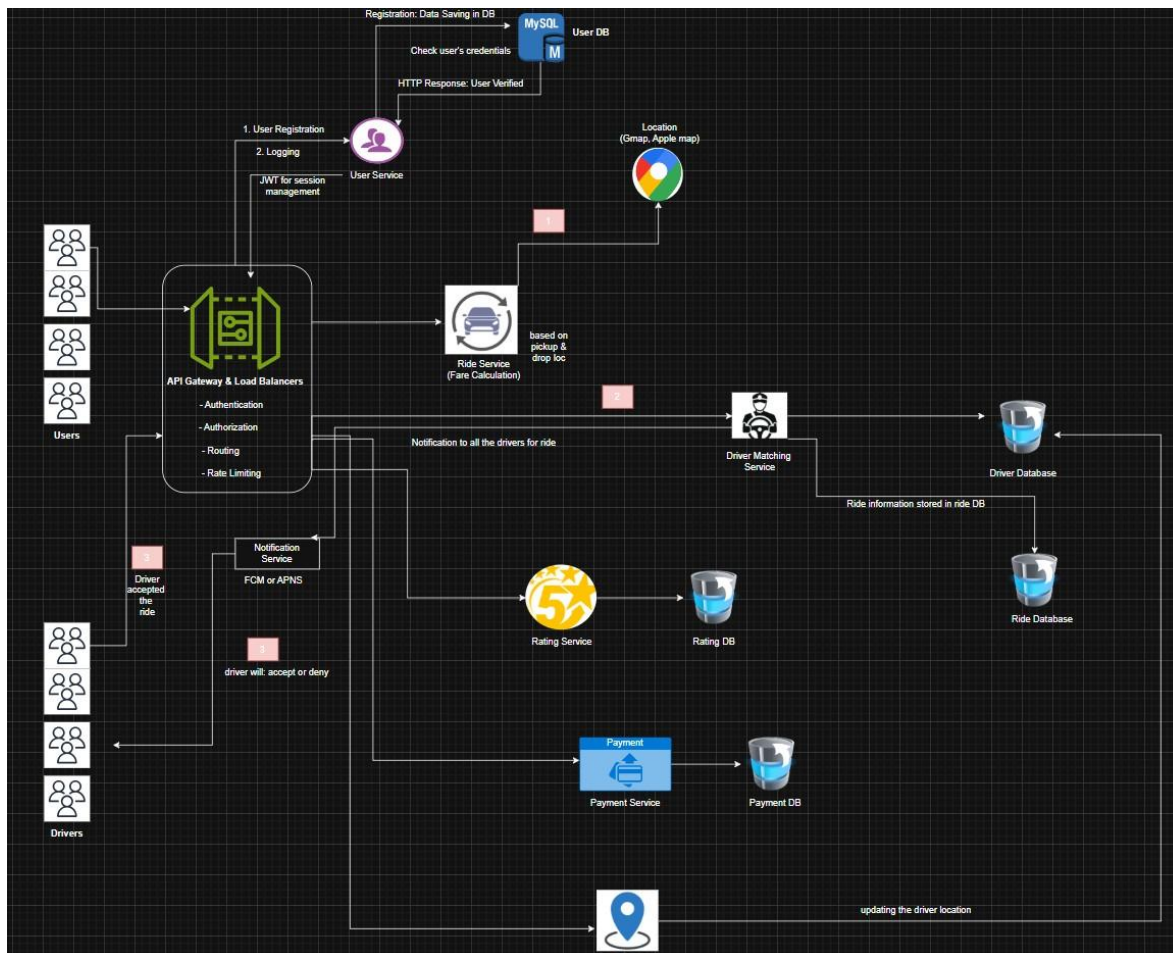
## Security

1. Secure authentication (JWT-based).
2. Secure payment processing.
3. Data encryption using HTTPS/TLS.

## High Availability

1. Load balancing across servers.
2. Fault tolerance and redundancy mechanisms.

## Required System Design-





# DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

## Outcome / Result -

The ride-sharing system was successfully designed with key features like ride booking, driver matching, tracking, and payments, ensuring scalability, low latency, and high availability for real- world usage.